CISC 3130 - Music Player Homework

Part A: Big-O Table

Method	Structures used	Big-O	Reason
addSong	Linked list, HashMap	O(1) avg.	Add at tail and store ID in map.
findSong	HashMap	O(1) avg.	Find song using its ID.
play	HashMap, stack, counts	O(1) avg.	Look up song and update history/count.
queueLater	HashMap, deque	O(1) avg.	Add to the back of deque.
queueNext	HashMap, deque	O(1) avg.	Add to the front of deque.
next	Deque, list, stack	O(k + 1) avg.	May skip k removed queued songs.
back	Stack, deque	O(k + 1) avg.	May skip k removed history songs.
removeSong	Linked list, HashMap	O(1) avg.	Map finds node; unlink it directly.
showPlaylist	Linked list	O(n)	Visit each song once.

Part B: Defend Two Choices

1. HashMap (Library)
I used a HashMap to find a song by its ID. It usually takes O(1) time to look up a song. If I used a normal array, I would need to search through songs one by one, which takes O(n). Java already has HashMap, so I did not need to build my own hash table.

2. Deque (Up Next)
I used ArrayDeque for the Up Next list because songs can be added to the front or the back. Both operations take O(1) time. With a normal array, adding a song at the front can mean moving the other elements. ArrayDeque is built into Java, so it is easier to use than making my own linked deque.